

SKILL WEEK – 2

SESSION – 3

TASK 1: To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality

```
keerthi_priya@LAPTOP-OSGV x + v
keerthi_priya@LAPTOP-OSGVCE7A:~$ nano session3_task1.c
keerthi_priya@LAPTOP-OSGVCE7A:~$ ^C
-bash: :s^C: substitution failed
keerthi_priya@LAPTOP-OSGVCE7A:~$ gcc session3_task1.c -o session3_task1
keerthi_priya@LAPTOP-OSGVCE7A:~$ ./session3_task1
Session 3 - Command History Program
Type commands and press Enter.
Use UP/DOWN arrows to recall commands.
Type exit to quit.

shell>
keerthi_priya@LAPTOP-OSGVCE7A:~$ |
```

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <termios.h>

#define MAX_HISTORY 10
#define MAX_LENGTH 100

char history[MAX_HISTORY][MAX_LENGTH];
int history_count = 0;

void enable_raw_mode(struct termios *original)
{
    struct termios raw;

    tcgetattr(STDIN_FILENO, original);
    raw = *original;

    raw.c_lflag &= ~(ICANON | ECHO);
    raw.c_cc[VMIN] = 1;
    raw.c_cc[VTIME] = 0;

    tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw);
}

void disable_raw_mode(struct termios *original)
{
    tcsetattr(STDIN_FILENO, TCSAFLUSH, original);
}

void store_history(char *command)
{
    if (strlen(command) == 0)
        return;

    if (history_count < MAX_HISTORY)
    {
        strcpy(history[history_count], command);
        history_count++;
    }
    else
    {
        for (int i = 1; i < MAX_HISTORY; i++)
        {
```

```
keerthi_priya@LAPTOP-OSGV x + v
GNU nano 8.7.1

if (history_count < MAX_HISTORY)
{
    strcpy(history[history_count], command);
    history_count++;
}
else
{
    for (int i = 1; i < MAX_HISTORY; i++)
    {
        strcpy(history[i - 1], history[i]);
    }

    strcpy(history[MAX_HISTORY - 1], command);
}
}

void clear_line(char *buffer, int *length)
{
    while (*length > 0)
    {
        printf("\b \b");
        (*length)--;
    }

    buffer[0] = '\0';
}

int main()
{
    struct termios original;

    char buffer[MAX_LENGTH];
    int length = 0;
    int history_index = history_count;

    printf("Session 3 - Command History Program\n");
    printf("Type commands and press Enter.\n");
    printf("Use UP/DOWN arrows to recall commands.\n");
    printf("Type exit to quit.\n\n");

    while (1)
    {
        printf("shell> ");
        fflush(stdout);
    }
}
```

```
keerthi_priya@LAPTOP-OSGV x + v
GNU nano 8.7.1 session

printf("Type exit to quit.\n\n");

while (1)
{
    printf("shell> ");
    fflush(stdout);

    buffer[0] = '\0';
    length = 0;
    history_index = history_count;

    enable_raw_mode(&original);

    while (1)
    {
        char ch;

        read(STDIN_FILENO, &ch, 1);

        /* Enter key */
        if (ch == '\n' || ch == '\r')
        {
            printf("\n");
            break;
        }

        /* Backspace */
        else if (ch == 127 || ch == 8)
        {
            if (length > 0)
            {
                length--;
                buffer[length] = '\0';

                printf("\b \b");
                fflush(stdout);
            }
        }

        /* Escape sequence */
        else if (ch == 27)
        {
            char sequence[2];

            read(STDIN_FILENO, &sequence[0], 1);
            read(STDIN_FILENO, &sequence[1], 1);
        }
    }
}
```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1 session

read(STDIN_FILENO, &sequence[0], 1);
read(STDIN_FILENO, &sequence[1], 1);

/*
ESC [ A = Up Arrow
ESC [ B = Down Arrow
*/

if (sequence[0] == '[' && sequence[1] == 'A')
{
    /* Navigate previous commands */
    if (history_count > 0 && history_index > 0)
    {
        history_index--;

        clear_line(buffer, &length);

        strcpy(buffer, history[history_index]);
        length = strlen(buffer);

        printf("%s", buffer);
        fflush(stdout);
    }
}

else if (sequence[0] == '[' && sequence[1] == 'B')
{
    /* Navigate next commands */
    if (history_index < history_count - 1)
    {
        history_index++;

        clear_line(buffer, &length);

        strcpy(buffer, history[history_index]);
        length = strlen(buffer);

        printf("%s", buffer);
        fflush(stdout);
    }
    else if (history_index == history_count - 1)
    {
        history_index++;

        clear_line(buffer, &length);
    }
}
```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1

        fflush(stdout);
    }
    else if (history_index == history_count - 1)
    {
        history_index++;

        clear_line(buffer, &length);
    }
}

/* Normal characters */
else if (ch >= 32 && ch <= 126)
{
    if (length < MAX_LENGTH - 1)
    {
        buffer[length++] = ch;
        buffer[length] = '\0';

        putchar(ch);
        fflush(stdout);
    }
}

disable_raw_mode(&original);

/* Store command in history */
store_history(buffer);

/* Test exit */
if (strcmp(buffer, "exit") == 0)
{
    printf("Exiting shell...\n");
    break;
}

if (strlen(buffer) > 0)
{
    printf("Command entered: %s\n", buffer);
}

return 0;
}
```

TASK 2: To Allocate Buffers Dynamically, Resize Arrays, Prevent Buffer Overflow, Ylanage Linked Lists, Release Memory Correctly, Verify with Valgrind

```
keerthi_priya@LAPTOP-OSGV × + ▾
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define INITIAL_SIZE 5
#define MAX_INPUT 100

struct Node
{
    int data;
    struct Node *next;
};

int main()
{
    int *array;
    int size = INITIAL_SIZE;
    int count = 0;
    char input[MAX_INPUT];

    /* Allocate buffer dynamically */
    array = (int *)malloc(size * sizeof(int));

    if (array == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    printf("Session 3 - Dynamic Memory Management\n");
    printf("Enter integers (enter -1 to stop):\n");

    while (1)
    {
        printf("Enter value: ");

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        /* Prevent buffer overflow */
    }
}
```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1

/* Prevent buffer overflow */
if (strchr(input, '\n') == NULL)
{
    int ch;

    while ((ch = getchar()) != '\n' && ch != EOF);

    printf("Input too long. Try again.\n");
    continue;
}

int value;

if (sscanf(input, "%d", &value) != 1)
{
    printf("Invalid input. Enter an integer.\n");
    continue;
}

if (value == -1)
    break;

/* Resize array when full */
if (count == size)
{
    size *= 2;

    int *temp = (int *)realloc(array, size * sizeof(int));

    if (temp == NULL)
    {
        printf("Memory reallocation failed.\n");
        free(array);
        return 1;
    }

    array = temp;

    printf("Array resized to %d elements.\n", size);
}
```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1

    printf("Array resized to %d elements.\n", size);
}

array[count++] = value;
}

printf("\nArray elements:\n");
for (int i = 0; i < count; i++)
{
    printf("%d ", array[i]);
}

printf("\n");

/* Manage linked list */
struct Node *head = NULL;
struct Node *tail = NULL;

for (int i = 0; i < count; i++)
{
    struct Node *newNode =
        (struct Node *)malloc(sizeof(struct Node));

    if (newNode == NULL)
    {
        printf("Linked list memory allocation failed.\n");
        free(array);
        return 1;
    }

    newNode->data = array[i];
    newNode->next = NULL;

    if (head == NULL)
    {
        head = newNode;
        tail = newNode;
    }
    else
```

```
keerthi_priya@LAPTOP-OSGV × + ▾
GNU nano 8.7.1
{
    head = newNode;
    tail = newNode;
}
else
{
    tail->next = newNode;
    tail = newNode;
}
}

printf("\nLinked List:\n");

struct Node *current = head;

while (current != NULL)
{
    printf("%d -> ", current->data);
    current = current->next;
}

printf("NULL\n");

/* Release linked list memory */
current = head;

while (current != NULL)
{
    struct Node *temp = current;
    current = current->next;
    free(temp);
}

/* Release dynamically allocated array */
free(array);

printf("\nAll dynamically allocated memory released.\n");

return 0;
}
```

```
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ gcc session3_task2.c -o session3_task2
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ ./session3_task2
Session 3 - Dynamic Memory Management
Enter integers (enter -1 to stop):
Enter value: 10
Enter value: 20
Enter value: 30
Enter value: 40
Enter value: 50
Enter value: 60
Array resized to 10 elements.
Enter value: -1

Array elements:
10 20 30 40 50 60

Linked List:
10 -> 20 -> 30 -> 40 -> 50 -> 60 -> NULL

All dynamically allocated memory released.
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ |
```

Session-4

Task 1: To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_TOKENS 50
#define MAX_TOKEN_LENGTH 50

typedef struct {
    char value[MAX_TOKEN_LENGTH];
} Token;

int main() {
    char input[500];
    Token tokens[MAX_TOKENS];
    int token_count = 0;

    printf("Session 4 - Tokenization\n");
    printf("Enter a command: ");
    fgets(input, sizeof(input), stdin);

    int i = 0;

    while (input[i] != '\0' && input[i] != '\n') {
        /* Skip whitespace */
        while (isspace((unsigned char)input[i])) {
            i++;
        }

        if (input[i] == '\0' || input[i] == '\n')
            break;

        /* Check token limit */
        if (token_count >= MAX_TOKENS) {
            printf("Error: Too many tokens.\n");
            break;
        }

        int j = 0;

        /* Read characters until whitespace or delimiter */
        while (input[i] != '\0' &&
            !isspace((unsigned char)input[i]) &&
            input[i] != '|') {
```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
    i++;
}

tokens[token_count].value[j] = '\0';

if (j > 0)
    token_count++;

/* Handle delimiters */
if (input[i] == '|') {
    if (token_count >= MAX_TOKENS) {
        printf("Error: Too many tokens.\n");
        break;
    }

    tokens[token_count].value[0] = input[i];
    tokens[token_count].value[1] = '\0';

    token_count++;
    i++;
}

printf("\nTokens identified:\n");

if (token_count == 0) {
    printf("No tokens found.\n");
    return 0;
}

for (int k = 0; k < token_count; k++) {
    printf("Token %d: [%s]\n", k + 1, tokens[k].value);
}

printf("\nToken stream validation:\n");

int valid = 1;

for (int k = 0; k < token_count; k++) {
    if (strlen(tokens[k].value) == 0) {
        valid = 0;
    }
}

if (valid)
    printf("Token stream is valid.\n");
else
    printf("Token stream is invalid.\n");

return 0;
}
```

```
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ gcc session4_task1.c -o session4_task1
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ ./session4_task1
Session 4 - Tokenization
Enter a command: ls -l | grep txt

Tokens identified:
Token 1: [ls]
Token 2: [-l]
Token 3: [|]
Token 4: [grep]
Token 5: [txt]

Token stream validation:
Token stream is valid.
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ |
```

TASK 2: To Design Parser Logic, Generate Parse Trees, Validate Syntax, Detect Errors, Handle Empty Commands, Produce Execution Structures

```
keerthi_priya@LAPTOP-OSGV  ×  +  ∨
GNU nano 8.7.1
#include <stdio.h>
#include <string.h>

#define MAX_TOKENS 50

int main() {
    char input[500];
    char *tokens[MAX_TOKENS];
    int token_count = 0;
    int valid = 1;

    printf("Session 4 - Parser\n");
    printf("Enter a command: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    /* Handle empty command */
    if (strlen(input) == 0) {
        printf("Empty command detected.\n");
        return 0;
    }

    /* Tokenize input */
    char *token = strtok(input, " \t");

    while (token != NULL && token_count < MAX_TOKENS) {
        tokens[token_count++] = token;
        token = strtok(NULL, " \t");
    }

    printf("\nParse Structure:\n");

    if (token_count == 0) {
        printf("No tokens found.\n");
        return 0;
    }

    /*
     * Basic syntax validation:
     * A pipe cannot be the first or last token.
    */
}
```

```
keerthi_priya@LAPTOP-OSGV  ×  +  ∨
GNU nano 8.7.1
/* Two pipes cannot occur consecutively.
 */
for (int i = 0; i < token_count; i++) {
    if (strcmp(tokens[i], "|") == 0) {

        if (i == 0 || i == token_count - 1) {
            valid = 0;
            printf("Syntax Error: Invalid pipe position.\n");
        }

        if (i > 0 && strcmp(tokens[i - 1], "|") == 0) {
            valid = 0;
            printf("Syntax Error: Consecutive pipes detected.\n");
        }
    }
}

if (!valid) {
    printf("\nParsing failed.\n");
    return 1;
}

/* Display execution structure */
printf("Command: %s\n", tokens[0]);

if (token_count > 1) {
    printf("Arguments:\n");

    for (int i = 1; i < token_count; i++) {
        if (strcmp(tokens[i], "|") != 0) {
            printf("  Argument %d: %s\n", i, tokens[i]);
        }
    }
}

printf("\nParse tree / execution structure:\n");
printf("ROOT\n");
printf("  └─ COMMAND: %s\n", tokens[0]);

for (int i = 1; i < token_count; i++) {
    if (strcmp(tokens[i], "|") == 0) {
        printf("    └─ PIPE\n");
    } else {
        printf("    └─ ARGUMENT: %s\n", tokens[i]);
    }
}

printf("\nSyntax validation: SUCCESS\n");
printf("Execution structure generated successfully.\n");

return 0;
}
```

```
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ nano session4_task2.c
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ gcc session4_task2.c -o session4_task2
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ ./session4_task2
Session 4 - Parser
Enter a command: ls -l | grep txt

Parse Structure:
Command: ls
Arguments:
  Argument 1: -l
  Argument 3: grep
  Argument 4: txt

Parse tree / execution structure:
ROOT
└─ COMMAND: ls
  └─ ARGUMENT: -l
    └─ PIPE
      └─ ARGUMENT: grep
        └─ ARGUMENT: txt

Syntax validation: SUCCESS
Execution structure generated successfully.
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ |
```